Reliability-Based Fleet Sizing under Stochastic Demand: A Comparative Analysis Across Several Operational Contexts

Martijn Liefrink (613067)
Eline de Wit (585936)
Roos Dees (777017)
Tom Picelli (580265)

March 18, 2026

Executive Summary

The Royal Netherlands Army commissioned TNO to investigate the logistical resupply of combat units during military missions. Specifically, the Army needs to ensure that essential commodities such as food, water, fuel, and ammunition reach frontline units without interruption, while deploying as few trucks as possible. This research therefore addresses two questions:

1. What is the minimum number of trucks required to prevent stock-outs during a military mission over a planning horizon of 10 days?
2. How does the required fleet size evolve as the mission duration increases?

To answer these questions, both deterministic and stochastic demand settings are examined using a total of five solution methods, compared across a range of problem sizes, operational periods, and network configurations. Comparing the two models directly quantifies the price of uncertainty - that is, the additional fleet size required solely due to the unpredictability of future demand.

The results reveal a substantial difference between deterministic and stochastic demand assumptions. For a mission duration of ten days, the deterministic model requires a minimum of 101 trucks to prevent stock-outs. When stochastic demand is considered at a 95% service level, the best-performing method requires a minimum fleet size of 154 trucks — an increase of over 50%. This gap illustrates the significant logistical cost of demand uncertainty and highlights the practical risk of relying solely on deterministic planning. Lowering the service level from 95% to 90% reduces the required fleet size from 154 to 146 trucks, illustrating the sensitivity of fleet capacity to the chosen service level requirement.

Furthermore, we observe that as the operational period increases, the minimum number of trucks to prevent stock-outs eventually stabilizes given deterministic demand, at 153 trucks. This result shows considerable relevance for military planning, since mission durations are often unknown in advance. Beyond instance size, different FSC configurations also influence the required fleet size. In the setting with known demand, around 4–6% additional trucks are needed to cover more dispersed network configurations, whereas this percentage grows to 10–14% under uncertain demand.

While these findings provide actionable guidance for the Royal Netherlands Army, several aspects of the problem remain outside the scope of this study and warrant further investigation. For future research, it is recommended to investigate the use of multiple truck types with different capacities, particularly in light of new vehicles entering the arsenal of the Royal Netherlands Army. Furthermore, future models could incorporate dynamic battlefield conditions in which operational areas shift due to tactical advances or retreats. Such developments may significantly affect the demand for essential supplies and therefore influence logistical requirements.

1 Introduction

Military operations depend critically on reliable and timely logistics. Essential commodities such as food, water, fuel, and ammunition are consumed daily by combat units, and shortages at operational locations can put personnel at risk and directly undermine mission effectiveness. This research addresses the problem of determining the minimum number of trucks required to resupply dispersed combat units over a finite planning horizon, without causing stock-outs at any operational location.

This problem carries significant practical relevance. As emphasized by TNO's Defense, Safety & Security mission, security and safety cannot be taken for granted, and resilient operational systems are essential. Developing quantitative planning tools that balance these concerns is therefore of direct operational value to the Royal Netherlands Army. Beyond the military context, this research contributes to the broader academic field of fleet sizing and vehicle routing under uncertainty, which has wide applications in humanitarian logistics and critical infrastructure supply chains.

We model the resupply problem as a multi-period fleet sizing problem over a network consisting of a main supply center (MSC), multiple forward supply centers (FSCs), and dispersed operating units (OUs). The goal is to find the minimum fleet size that supports a feasible shipment and allocation plan while respecting transportation and inventory constraints. Throughout this research, we examine both deterministic and stochastic demand, vary the planning horizon, and assess different network configurations and service levels.

For the deterministic setting, we develop an exact solver using a commercial branch-and-cut (B&C) algorithm and benchmark it against a Multi-Cut Benders Decomposition (MCBD). Benders decomposition is a classical technique for solving large-scale mixed-integer optimization problems by splitting them into a master problem and a series of easier subproblems. This is illustrated by Bidhandi et al. (2009), who demonstrate the decomposition's effectiveness for solving a static planning problem.

For the stochastic setting, demand uncertainty is handled using a Sample Average Approximation (SAA) method embedded in a rolling horizon framework. This approach has been widely studied within the literature to model stochasticity, where an illustrative example is given by Mancilla and Storer (2012), who solve a dynamic scheduling and sequencing problem by means of SAA. This approach is benchmarked against a Stochastic Multi-Cut Benders Decomposition (SMCBD). You and Grossmann (2013) suggest a multi-cut variant to process transportation planning and freight rate uncertainty, where adding a cut per scenario per iteration significantly increases computational efficiency compared to the standard single-cut approach. Similarly, Cardoso et al. (2014) develop an accelerated version for the design of sustainable medical scheduling networks, demonstrating its value for large-scale problems with uncertain parameters.

Our computational results show that for a ten-day mission, a minimum of 101 trucks is required under deterministic demand, which increases over 50% under stochastic demand when a 95% service level is considered. This illustrates the cost of demand uncertainty. Dispersed variants of the network need more trucks: 4-6% above the base case for the deterministic setting, and 10-14% for the stochastic setting. Furthermore, we observe that the minimum fleet size stabilizes as the operational period increases in the deterministic context for the standard instance.

The remaining structure of the report is as follows. Chapter 2 outlines the problem definition in detail and formalizes the mathematical framework with a MILP in Chapter 3. The proposed solution methods for the deterministic setting are outlined in Chapter 4. For the stochastic setting, Chapter 5.1 shows the employed solution approaches. In Chapter 6 the computational setup of the experiments is presented. Chapter 7 reports and discusses the computational results. To conclude, Chapter 8 summarizes the key findings and provides suggestions for future research.

2 Problem Description

This research considers a military resupply planning problem in which OUs must be supplied daily with three types of commodities: food and water, fuel, and ammunition. The objective is to determine the minimum number of trucks required to guarantee a certain service level at the OUs.

The logistics network consists of a MSC, several FSCs, and a set of OUs that consume supplies on a daily basis. Supplies arrive at the MSC in large quantities and are repackaged into Combat Configuration Loads (CCLs). Each CCL contains a predefined combination of commodities and is assigned to a specific OU. The combination of its load composition and assigned destination determines the CCL type. Once a CCL is assembled and labeled, this allocation decision is irreversible. From the MSC, CCLs are transported to either the FSCs or directly linked OUs. The FSCs act as intermediate storage and distribution points. They subsequently supply the OUs for whom they are responsible.

The network includes four types of OUs: Vust (fire support), GN cie (engineering company), Painf cie (armoured infantry company), and AT cie (anti-tank company). The resulting network structure for the base instance is illustrated in Figure 1.

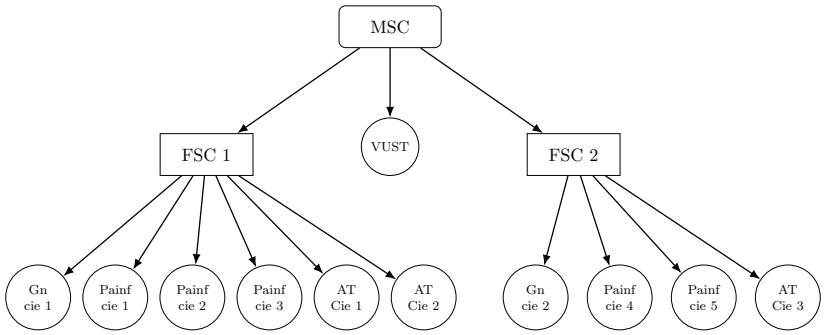

Figure 1: Logistics network for the Normal instance.

The resupply problem is evaluated for some chosen time horizon. During this time period, resupply takes place once on a daily basis. We require no OU runs out of any commodity before the next resupply moment. Each OU has a daily demand and a limited storage capacity for each commodity. However, at the beginning of the planning period each OU stocks its full storage capacity. Deliveries occur in integer numbers of CCLs. Inventory levels evolve according to a standard stock balance. Demand is assumed to occur at the beginning of each day, while replenishment arrives at the end of the day. Therefore, inventory levels at the start of each period must be sufficient to satisfy daily demand. This demand may be modeled as deterministic or stochastic. In the deterministic setting, the daily demand per commodity for each OU is assumed to be known with certainty over the entire planning horizon. In the stochastic setting, daily demand is uncertain and drawn from predetermined distributions, each describing a possible realization of demand per commodity over time. In the stochastic case, the fleet size must guarantee a predefined service level. Specifically, the number of trucks must be sufficient to prevent stock-outs for a required percentage of the considered demand scenarios. While delivery decisions may adapt to realized demand over time, the fleet size itself is fixed prior to the start of the planning horizon.

To conclude, the decision problem is to determine the minimum fleet size such that a feasible daily resupply schedule exists over the entire planning horizon. Feasibility requires that, for each day, sufficient CCLs are transported from warehouses to OUs to satisfy demand while respecting vehicle capacity constraints, warehouse flow conservation, and OU storage limits.

3 Mathematical Problem Formulation

To formalize the problem, we first introduce the necessary notation for sets, parameters, and decision variables, which is subsequently used to state the problem as a Mixed Integer Linear Programming (MILP) formulation.

Sets

- L: set of all locations (FSCs, MSC and OUs).
- $F \subset L$: set of FSCs.
- $K \subset L$: set of OUs.
- A: the set of directed arcs (i, j) for $i, j \in L$ in the network.
- T: time horizon, indexed by t, where $t \in \{1, \ldots, T\}$.
- U: set of combinations of CCL and OU types indexed by u, where $u \in U$. U_k indicates the set of CCLs that can be assigned to OU $k \in K$.
- P: set of commodities indexed by p.

Parameters

- c_f: the maximum capacity as number of CCLs in FSC $f \in F$.
- $c_{k,p}$: the maximum capacity of commodity $p \in P$ in OU $k \in K$.
- $q_{f,u}$: initial storage in location $f \in F$ for CCL type $u \in U$.
- $d_{k,p,t}$: demand of commodity $p \in P$ at OU $k \in K$ at time $t \in T$.
- $b_{u,p}$: the content of commodity $p \in P$ in CCL type $u \in U$.

Decision Variables

- $x_{i,j,u,t}$: cumulative flow of type $u \in U$ on arc $(i, j) \in A$ up to, and including, time $t \in T$.
- $I_{f,u,t}$: inventory of FSC $f \in F$ as number of CCLs of type $u \in U$ on day $t \in T$.
- $I_{k,p,t}$: inventory of location $k \in K$ as weight of commodity $p \in P$ at day $t \in T$.
- z_i: the number of trucks stationed at location $i \in L \setminus K$.

Using this notation, the deterministic problem is formulated as follows.

MILP Formulation

$$\min \quad \sum_{i \in L \setminus K} z_i \tag{1}$$

$$\text{s.t.} \quad z_i \geq \sum_{j:(i,j) \in A} \sum_{u \in U} (x_{i,j,u,t} - x_{i,j,u,t-1}) \qquad \forall\, i \in L \setminus K,\ t \in T \tag{2}$$

$$x_{i,j,u,t-1} \leq x_{i,j,u,t} \qquad \forall\, (i,j) \in A,\ u \in U,\ t \in T \tag{3}$$

$$I_{f,u,t} = I_{f,u,0} + \sum_{i:(i,f) \in A} x_{i,f,u,t} - \sum_{j:(f,j) \in A} x_{f,j,u,t} \qquad \forall\, f \in F,\ u \in U,\ t \in T \tag{4}$$

$$I_{k,p,t} = I_{k,p,0} + \sum_{i \in L \setminus K} \sum_{u \in U_k} x_{i,k,u,t} b_{u,p} - \sum_{m=0}^{t} d_{k,p,m} \qquad \forall\, k \in K,\ p \in P,\ t \in T \tag{5}$$

$$I_{k,p,t} \geq d_{k,p,t+1} \qquad \forall\, k \in K,\ p \in P,\ t \in T \tag{6}$$

$$I_{k,p,t} \leq c_{k,p} \qquad \forall\, k \in K,\ p \in P,\ t \in T \tag{7}$$

$$\sum_{u \in U} I_{f,u,t} \leq c_f \qquad \forall\, f \in F,\ t \in T \tag{8}$$

$$I_{k,p,0} = c_{k,p} \qquad \forall\, k \in K,\ p \in P \tag{9}$$

$$I_{f,u,0} = q_{f,u} \qquad \forall\, f \in F,\ u \in U \tag{10}$$

$$I_{k,p,t} \geq 0 \qquad \forall\, k \in K,\ p \in P,\ t \in T \tag{11}$$

$$I_{f,u,t} \geq 0 \qquad \forall\, f \in F,\ u \in U,\ t \in T \tag{12}$$

$$x_{i,j,u,t} \in \mathbb{Z}_+ \qquad \forall\, (i,j) \in A,\ u \in U,\ t \in T \tag{13}$$

$$z_i \in \mathbb{Z}_+ \qquad \forall\, i \in L \setminus K \tag{14}$$

In the introduced MILP formulation, the capacitated resupply problem is characterized by the following objective function and constraints. The objective (1) seeks to minimize the total number of trucks. Constraint (2) bounds the number of trucks by the maximum flow over outgoing arcs for each warehouse $i \in L \setminus K$ for each time period $t \in T$, since each unit of flow requires one truck. Furthermore, constraint (3) ensures continuity in the cumulative flows for each successive time period $t \in T$. Constraints (4) and (5) represent the inventory balance equations for the FSCs $f \in F$ and OUs $k \in K$, respectively. Constraint (6) enforces the absence of stock-outs in each period. Storage capacity limitations at the OUs and FSCs are imposed by Constraints (7) and (8). The initial inventory levels are specified by Constraints (9) and (10). Finally, Constraints (11) and (12) ensure non-negativity of FSC and OU inventories, Constraint (13) enforces integer flow, and constraint (14) restricts the number of trucks to be non-negative integer values.

4 Solution Methods for Deterministic Demand

In this section, we consider solution methods for the case in which demand is known and constant over all time periods. We introduce the following three approaches to solve the MILP in this static setting. First, we solve the MILP using a commercial B&C solver. we additionally develop a cumulative flow formulation (MCBD) and a per-period flow formulation, an Alternative Multi-Cut Benders Decompostion (AMCBD), to investigate whether decomposition-based approaches offer computational advantages for larger instances. The performance of these three methods is compared in Section 7.1 in terms of solution quality and computational efficiency.

4.1 Branch-and-cut

The MILP, outlined in Chapter 3, with deterministic demand is solved using Gurobi's B&C algorithm. All models are implemented in Java and solved using Gurobi version 12.0.0. All parameters are kept at their default values.

4.2 Benders Decomposition

The second algorithm we introduce is a Multi-Cut Benders Decomposition (MCBD). Benders decompositions are known to be highly effective for solving large-scale MILP problems when the Subproblem (SP) can be solved efficiently, as described by Chuong et al. (2025), often with a Linear Programming (LP) formulation. A key challenge with this approach is that it may require many iterations before the algorithm converges. Beheshti Asl and MirHassani (2018) describe how exploiting the separable structure of the SP allows generating multiple Benders cuts per iteration instead of a single aggregated cut, and shows the algorithm can be accelerated by adding multiple cuts to the master problem in each iteration. This typically improves the convergence of the algorithm.

It is shown that the multi-cut approach improves computational performance for the problem at hand. In this study, we consider two formulations of the algorithm: the first version is based on cumulative flow and secondly we discuss a flow-based variant per time period $t \in T$.

In the Benders decomposition, we decompose the MILP problem, introduced in Chapter (3), into a Master Problem (MP) and a SP. The MP contains the integer truck z_i and flow decisions $x_{i,j,u,t}$, while the SP includes the inventory balance and capacity constraints. By separating the integer and continuous variables, the SP reduces to a LP problem, which can be solved efficiently. If the SP is infeasible, a feasibility cut is added to the MP. Once the SP is feasible, the algorithm has converged to the optimal solution. The MP is defined as follows:

Master Problem

$$\min \sum_{i \in L \setminus K} z_i \tag{15}$$

$$\text{s.t.} \quad z_i \geq \sum_{j:(i,j)\in A} \sum_{u \in U} (x_{i,j,u,t} - x_{i,j,u,t-1}) \qquad \forall\, i \in L \setminus K,\ t \in T \tag{16}$$

$$x_{i,j,u,t-1} \leq x_{i,j,u,t} \qquad \forall\, (i,j) \in A,\ u \in U,\ t \in T \tag{17}$$

$$x_{i,j,u,t} \in \mathbb{Z}_+ \qquad \forall\, (i,j) \in A,\ u \in U,\ t \in T \tag{18}$$

$$z_i \in \mathbb{Z}_+ \qquad \forall\, i \in L \setminus K \tag{19}$$

As stated, the MP finds a solution for the number of trucks $\bar{z}_i$ and flows $\bar{x}_{i,j,u,t}$ over all arcs $(i,j) \in A$, for all CCL and OU types $u \in U$, and for all time periods $t \in T$. Fixing these values, the resulting

SP is defined as follows:

Subproblem

$$\min \quad 0 \tag{20}$$

$$I_{f,u,t} = I_{f,u,0} + \sum_{i:(i,f)\in A} \bar{x}_{i,f,u,t} - \sum_{j:(f,j)\in A} \bar{x}_{f,j,u,t} \qquad \forall\, f \in F,\ u \in U,\ t \in T \tag{21}$$

$$I_{k,p,t} = I_{k,p,0} + \sum_{i\in L\setminus K} \sum_{u\in U_k} \bar{x}_{i,k,u,t} b_{u,p} - \sum_{m=1}^{t} d_{k,p,m} \qquad \forall\, k \in K,\ p \in P,\ t \in T \tag{22}$$

$$\text{Constraint (6) - (12)} \tag{23}$$

Since the SP is formulated as a feasibility problem with objective value zero, infeasibility of the SP implies that no inventory levels exist that satisfy the balance and capacity constraints for the given MP solution $(\bar{z}, \bar{x})$. In standard form, the above MILP formulations can be rewritten as follows:

$$\min \quad c^T z, \tag{24}$$

$$AI + Bx + Cz \geq b, \tag{25}$$

$$x \in \mathbb{Z}_+, \tag{26}$$

$$z \in \mathbb{Z}_+, \tag{27}$$

$$I \geq 0, \tag{28}$$

where A, B, and C are the coefficient matrix corresponding to their associated variable vectors. To derive the feasibility cut we apply Farkas' lemma, as presented by Bartl (2007). Let the constraints of the SP be written as:

$$AI \leq b(\bar{x}), \quad I \geq 0,$$

where I denotes the inventory variables. Farkas' lemma states that if the SP is infeasible, then there exists a dual ray π such that:

$$\pi^\top A = 0, \qquad \pi^\top b(\bar{x}) < 0.$$

With this, a feasibility cut can be formulated as follows:

$$\pi(b - Ax) \leq 0 \tag{29}$$

For the feasibility cut, we first define the dual multipliers of the SP. Let $\lambda^{iter}_{f,u,t}$, $\beta^{iter}_{k,p,t}$, $\gamma^{iter}_{k,p,t}$, $\delta^{iter}_{k,p,t}$ and $\eta^{iter}_{f,t}$ be the dual multipliers for constraint (21) - (22) and (6) - (8) respectively. The resulting feasibility cut added to the MP is

$$\begin{aligned}
&\sum_{f\in F}\sum_{u\in U}\sum_{t\in T} \lambda^{iter}_{f,u,t}(q_{f,u} + \sum_{i:(i,f)\in A} x_{i,f,u,t} - \sum_{j:(f,j)\in A} x_{f,j,u,t}) \\
&+ \sum_{k\in K}\sum_{p\in P}\sum_{t\in T} \beta^{iter}_{k,p,t}(c_{k,p} + \sum_{i\in L\setminus K}\sum_{u\in U_k} x_{i,k,u,t} b_{u,p} - \sum_{m=1}^{t} d_{k,p,m}) \\
&+ \sum_{k\in K}\sum_{p\in P}\sum_{t\in T} \delta^{iter}_{k,p,t} c_{k,p} - \gamma^{iter}_{k,p,t} d_{k,p,t+1} \\
&+ \sum_{f\in F}\sum_{t\in T} \eta^{iter}_{f,t} c_f \leq 0.
\end{aligned} \tag{30}$$

Because this cut is fairly general in the information it gives to the MP, the cut is weak and this may result in slow convergence of the algorithm. Similarly, the MP has a lot of decision variables that are weakly bounded, which may lead to slow convergence in the B&C algorithm used to solve the MP.

Having established the general decomposition framework, the two specific formulations considered in this study are presented below.

4.2.1 Alternative formulation

In the preceding section, the problem was formulated using cumulative deliveries per period. As mentioned earlier, an alternative and more intuitive formulation considers truck deliveries on a per-period basis rather than cumulatively. This modification replaces Constraints (2)–(5) in the original formulation with the following:

$$z_i \geq \sum_{j:(i,j)\in A} \sum_{u\in U} x_{i,j,u,t} \qquad \forall\, i \in L,\ t \in T \tag{31}$$

$$I_{f,u,t} = I_{f,u,t-1} - \sum_{j:(f,j)\in A} x_{f,j,u,t} + \sum_{i:(i,f)\in A} x_{i,f,u,t} \qquad \forall\, f \in F,\ u \in U,\ t \in T \tag{32}$$

$$I_{k,p,t} = I_{k,p,t-1} + \sum_{i:(i,k)\in A} \sum_{u\in U_k} x_{i,k,u,t} b_{u,p} - d_{k,p,t} \qquad \forall\, k \in K,\ p \in P,\ t \in T \tag{33}$$

The key distinction of this formulation is that the inventory state depends on that of the preceding period. Consequently, the SP can no longer be decomposed by time period. Moreover, Constraint (17) is removed from the MP, which weakens it and may lead to slower convergence. The effect of this modification is demonstrated numerically in Section 7.1.

4.2.2 Multi-Cut Benders Decomposition

For both the original and alternative formulation (AMCBD), we use a multi-cut approach to improve the efficiency of the Benders decomposition. This approach is inspired by the solution method used in You and Grossmann (2013), where a MCBD algorithm is used to solve a capacitated supply chain problem.

The decomposition exploits the fact that the SP is separable in time, location, and commodity. Consequently, for a solution to be feasible, the inventory constraints must be satisfied at each location $l \in L$, for each time period $t \in T$ and each commodity $p \in P$. Equivalently, this can be understood through the block-angular structure of the SP, which extends across all three of these dimensions. This yields $|T| \times (|F| + |K| \times |P|)$ independent SPs, each admitting its own feasibility cut. Since the feasibility cuts depend on the location $l \in L$ of the corresponding SP, distinct cut structures arise for warehouse and OU SPs. The following cuts are generated:

$$\sum_{u\in U} \lambda^{iter}_{f,u,t}(q_{f,u} + \sum_{i:(i,f)\in A} x_{i,f,u,t} - \sum_{j:(f,j)\in A} x_{f,j,u,t}) + \eta^{iter}_{f,t} c_f \leq 0, \quad \forall f \in F, t \in T, \tag{34}$$

$$\beta^{iter}_{k,p,t}(c_{k,p} + \sum_{i\in L\setminus K} \sum_{u\in U_k} x_{i,k,u,t} b_{u,p} - \sum_{m=1}^{t} d_{k,p,m}) + \delta^{iter}_{k,p,t} c_{k,p} - \gamma^{iter}_{k,p,t} d_{k,p,t+1} \leq 0, \quad \forall k \in K, p \in P, t \in T \tag{35}$$

The algorithm framework follows that of You and Grossmann (2013), and is given as follows:

Algorithm 1 Multi-cut Benders Decomposition Algorithm

Set iter $= 1$, LB $= -\infty$.
while there exists a SP that is not feasible **do**
    Solve the master problem with all feasibility cuts from previous iterations.
    Let the optimal objective value be ϕ^{iter} and the first-stage solution be X^{iter}.
    if $\phi^{\text{iter}} >$ LB **then**
        LB $\leftarrow \phi^{\text{iter}}$
    end if
    Solve the SPs with $\bar{x} = X^{\text{iter}}$.
    For all SPs, check if the SP is feasible.
    if SP is infeasible **then**
        Compute the dual rays of the SP $\lambda^{iter}_{f,u,t}$, $\beta^{iter}_{k,p,t}$, $\gamma^{iter}_{k,p,t}$, $\delta^{iter}_{k,p,t}$ and $\eta^{iter}_{f,t}$
        Add the feasibility cut (34) or (35) to the master problem.
    end if
    iter $\leftarrow$ iter $+ 1$
end while
Output: Optimal fleet size z_i and solution X^{iter} with objective value LB.

5 Solution Methods for Stochastic Demand

This chapter presents two solution methods for the case of uncertain demand. We introduce an approach which handles uncertainty by means of a SAA heuristic in combination with a rolling horizon. We compare these methods to a Stochastic Multi-Cut Benders Decomposition (SMCBD), which is an extension of the deterministic version introduced in Section 4.2.

5.1 Sample Average Approximation within a rolling horizon framework

The SAA rolling horizon approach consists of three interacting components. The inner layer, described in Section 5.1.1, determines at each decision epoch whether a feasible shipment plan exists for a given fleet size, by solving a short-horizon MILP over a set of sampled demand scenarios. The outer layer, described in Section 5.1.2, evaluates the true reliability of a given fleet size via an out-of-sample Monte Carlo simulation, providing an unbiased estimate of the service level achieved under the rolling horizon policy. Finally, the binary search procedure, described in Section 5.1.3, efficiently identifies the minimum fleet size that meets the required service level α by exploiting the monotonicity of reliability in the fleet size. Together, these three components form a nested procedure in which the binary search drives the fleet size selection, the outer layer evaluates each candidate fleet size, and the inner layer determines feasibility at each decision epoch.

5.1.1 The inner layer: rolling horizon SAA

The purpose of the inner layer is to determine, at each decision epoch, whether there exist shipment decisions that prevent stock-outs over a short look-ahead horizon under demand uncertainty. A demand scenario ξ is defined as the collection of all demand realizations for each operating unit $k \in \mathcal{K}$ and time period $t \in \mathcal{T}$:

$$\xi = \{d_{k,t}^{(\xi)} : k \in \mathcal{K},\ t \in \mathcal{T}\}.$$

Here, $d_{k,t}^{(\xi)}$ denotes the demand of operating unit k in period t under scenario ξ.

Consider time horizon T, divided into decision epochs $t = 1, \ldots, T$. At the start of each epoch t, all demand realizations are known up until $t-1$, and inventory levels $I_{k,p,t-1}$ and $I_{f,u,t-1}$ are observed. Future demand, however, is still uncertain. Rather than optimizing over the full horizon, a look-ahead window of length H_{SAA} is defined and we cover only periods $\tau = \{t, \ldots, t+H_{SAA}-1\}$. Within this shorter window, uncertainty is handled via SAA. To this end, we sample N_{SAA} independent demand scenarios and define the set $S_{\text{SAA}} = \{\xi^1, \ldots, \xi^{N_{\text{SAA}}}\}$. The MILP is then solved over the window $\tau = t, \ldots, t + H_{SAA} - 1$ simultaneously for all sampled scenarios.

Since demand differs per scenario, the inventory trajectories are not identical across scenarios. We therefore introduce scenario-dependent inventories $I_{k,p,\tau}^{(\xi)}$ and $I_{f,u,\tau}^{(\xi)}$, and enforce separate inventory balance equations for each $\xi \in S_{\text{SAA}}$:

$$I_{f,u,\tau}^{(\xi)} = I_{f,u,\tau-1}^{(\xi)} - \sum_{j:(f,j)\in A} x_{f,j,u,\tau} + \sum_{i:(i,f)\in A} x_{i,f,u,\tau} \quad \forall\, f \in F,\ u \in U,\ \tau \in [t, t{+}H_{SAA}{-}1],\ \xi \in S_{\text{SAA}} \tag{36}$$

$$I_{k,p,\tau}^{(\xi)} = I_{k,p,\tau-1}^{(\xi)} + \sum_{i:(i,k)\in A} \sum_{u\in U_k} x_{i,k,u,\tau} b_{u,p} - d_{k,p,\tau}^{(\xi)} \quad \forall\, k \in K,\ p \in P,\ \tau \in [t, t{+}H_{SAA}{-}1],\ \xi \in S_{\text{SAA}} \tag{37}$$

Equations (36) and (37) replace Constraints (4) and (5), respectively. Stock-outs are likewise prohibited for every τ in the window and every sampled scenario, replacing Constraints (6), (7), and (8):

$$I^{(\xi)}_{k,p,\tau} \geq d^{(\xi)}_{k,p,\tau} \qquad \forall\, k \in K,\ p \in P,\ \tau \in [t, t{+}H_{SAA}{-}1],\ \xi \in S_{\text{SAA}} \tag{38}$$

$$I^{(\xi)}_{k,p,\tau} \leq c_{k,p} \qquad \forall\, k \in K,\ p \in P,\ \tau \in [t, t{+}H_{SAA}{-}1],\ \xi \in S_{\text{SAA}} \tag{39}$$

$$\sum_{u \in U} I^{(\xi)}_{f,u,\tau} \leq c_f \qquad \forall\, f \in F,\ \tau \in [t, t{+}H_{SAA}{-}1],\ \xi \in S_{\text{SAA}} \tag{40}$$

The shipment decisions within the window are split into two stages. At the current epoch t, only one supply plan can be executed, so the decision must be identical across all scenarios — this is the *first-stage* decision, shown in blue in Figure 2. This restriction is incorporated in the MILP by adding Constraint 41.

$$x^{(\xi)}_{i,j,u,t} = x^{(\xi')}_{i,j,u,t} \quad \forall (i,j) \in A,\ u \in U,\ \forall \xi, \xi' \in S_{\text{SAA}}. \tag{41}$$

Shipment decisions for future periods $\tau = t{+}1, \ldots, t{+}H_{SAA}{-}1$ within the window are *second-stage* variables (purple in Figure 2): they are allowed to be scenario-dependent, since they serve only as planning variables that guide the current first-stage decision and are never directly executed.

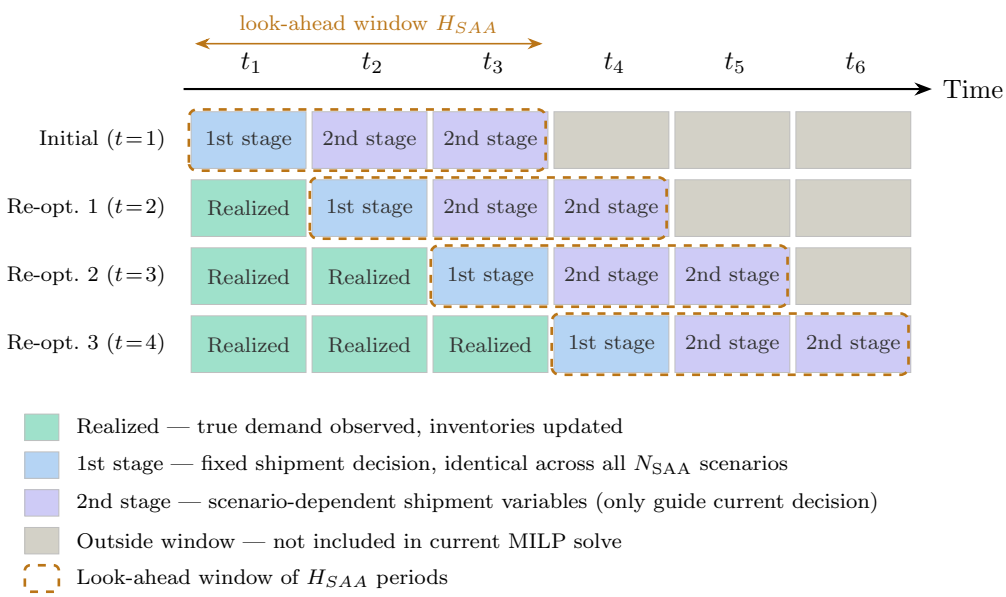

Figure 2: Illustration of the rolling horizon SAA re-optimization.

Once the MILP is solved, only the first-stage shipments at epoch t are implemented. The system then transitions to epoch $t+1$: true demand for period t is realized, inventories are updated (the green *Realized* cells in Figure 2), and the window advances to $\tau = t+1, \ldots, t+H_{SAA}$. In this way, we "roll" forward the horizon one period at a time, continuously incorporating updated information. A new set of N_{SAA} scenarios is sampled over the new window, and the process repeats until the end of the horizon.

5.1.2 The outer layer: reliability estimation

While the inner layer determines whether a feasible rolling horizon policy can be constructed for a given fleet size Z under sampled scenarios, however this does not yet guarantee the size attains the

required service level. The SAA inside the rolling horizon only considers a limited number N_{SAA} of scenarios at each epoch. Therefore, a separate evaluation step is required to assess the true performance of the fleet size under stochastic demand.

The evaluation is carried out by means of an out-of-sample Monte Carlo simulation. We generate N independent demand scenarios over the full horizon T. For each scenario, we simulate the entire procedure sequentially and obtain a shipment decision from solving the MILP for the current time period as described in Section 5.1.1. The resulting supply plan is implemented, demand for the current epoch t is realized according to the Monte Carlo scenario, and inventories are updated accordingly. We continue this process until the end of the horizon is reached or a stock-out occurs.

We classify each Monte Carlo scenario as a success if no stock-outs occur during the entire horizon. The reliability of fleet size Z is then given by the fraction of the number of successful Monte Carlo scenarios $N_{\text{succ}}(Z)$ divided by the total number of simulation runs N. Formally, we can write the empirical reliability $\hat{R}(Z)$ as follows:

$$\hat{R}(Z) = \frac{N_{\text{succ}}(Z)}{N}$$

This separate evaluation procedure therefore provides an unbiased estimate of the rolling horizon policy performance for fleet size Z. Importantly, the N demand scenarios used in this part are independent from the N_{SAA} scenarios inside the SAA optimization, to prevent overfitting to a particular sample of demand realizations.

5.1.3 Binary Search Methods: fleet size selection

To efficiently explore the search space, we apply two versions of the Binary Search (BS) method. First, we use a standard BS and later an extension of this approach, the Bayesian Binary Search (BBS), which can provide a more robust procedure in the presence of noisy or uncertain performance estimates.

For brevity, the SAA rolling horizon approach combined with the standard BS search and BBS are referred to as SAABS and SAABBS, respectively, throughout the remainder of this paper.

Classic Binary Search The outer layer provides, for a given fleet size Z, an estimate of the corresponding reliability level $\hat{R}(Z)$. The objective is to determine the minimum fleet size for which the required service level α is reached. Formally, we seek for the smallest Z^* such that:

$$\hat{R}(Z^*) \geq \alpha.$$

Evaluating the reliability for every possible fleet size is computationally expensive, since each evaluation requires a full Monte Carlo simulation combined with repeated rolling horizon optimizations. Instead, we exploit the monotonicity of the problem: increasing the fleet size cannot decrease the ability to prevent stock-outs. Hence, reliability is non-decreasing in Z. This monotonic property enables us to use a binary search procedure to find the minimum fleet size. We introduce a lower bound $Z_{\min}$ and upper bound $Z_{\max}$ on the fleet size. At each iteration, we evaluate the midpoint:

$$Z_{\text{mid}} = \left\lfloor \frac{Z_{\min} + Z_{\max}}{2} \right\rfloor.$$

We execute the nested procedure for Z_{mid} and obtain estimated reliability $\hat{R}(Z_{\text{mid}})$. If the reliability meets or exceeds the service level α, the fleet size is potentially larger than necessary and the upper bound is therefore updated to Z_{mid}. Otherwise, the fleet size is insufficient to meet the service level

and the lower bound is increased until $Z_{\text{mid}} + 1$. We continue this procedure until the bounds coincide. The resulting value is the smallest fleet size for which the estimated reliability satisfies the required service level. In this way, the binary search efficiently identifies the minimum fleet size while limiting the number of computationally expensive simulation evaluations.

Bayesian Binary Search Bayesian Binary Search (BBS) extends the standard BS by explicitly accounting for the uncertainty in the reliability estimate $\hat{R}(Z^*)$. Since each estimate is based on a finite sample of N Monte Carlo simulations, it exposes us to sampling noise. Therefore, an approach is adopted that models the true reliability as a random variable rather than treating $\hat{R}(Z^*)$ as a fixed deterministic value, as BS does, but model the true underlying distribution as a random variable, which updates its belief about this probability as simulation outcomes are observed. An overview of this conservative search method is outlined in Singh et al. (2024).

A Bernoulli success probability p_z is used to model the true reliability of Z, where $p_z = 1$ if the Monte Carlo run succeeds (no stock-out at OUs) and $p_z = 0$ if it fails. A Beta distribution is used to model the uncertainty about p_z, which is the natural conjugate of prior for binomial outcomes, as described by Wolfram Research, Inc. (2024). Initially, we use an uninformative prior $Beta(\sigma, \mu)$, where $\sigma = 1$ and $\mu = 1$. This assigns an equal probability to the binary outcome in $[0, 1]$ for p_z. Throughout the search procedure, we update the parameters after each batch of N simulation runs as follows:

$$\sigma_{\text{posterior}} = \sigma_{\text{prior}} + N_{\text{succ}}, \qquad \mu_{\text{posterior}} = \mu_{\text{prior}} + (N - N_{\text{succ}})$$

where N_{succ} denotes the number of successful runs out of all N simulation runs. A posterior distribution $p_Z \sim \text{Beta}(\sigma_{\text{posterior}}\mu_{\text{posterior}})$ is yielded, and this reflects both the prior belief and the observed outcome.

Fleet size Z is then feasible if the posterior probability of attaining service level α exceeds a confidence threshold γ.

$$P(p_Z \geq \alpha) = \int_{\alpha}^{1} \frac{x^{\sigma_{\text{posterior}}-1}(1-x)^{\mu_{\text{posterior}}-1}}{B(\sigma_{\text{posterior}},\, \mu_{\text{posterior}})}\, dx \geq \gamma$$

Afterward, the procedure continues equivalently to the BS, terminating as the bounds coincide. The key advantage over BS is that the feasibility decision incorporates posterior uncertainty rather than relying on a single noisy point estimate, which provides a more robust fleet size against sampling variability.

5.2 Stochastic Multi-Cut Benders Decomposition

For the SMCBD, a scenario-based uncertainty structure is adopted, analogously to the approach introduced in Section 5.1. Multiple formulations of this structure are possible, one of which is presented in this section. An alternative formulation is provided in Appendix A; however, no computational results are reported for this formulation, as it could not be solved within the scope of this study.

The implemented solution method for fleet size determination is a variation of the deterministic Benders decomposition introduced in Section 4. A key challenge arising from demand uncertainty in this problem is the sequential realization of uncertainty, while the fleet size must be minimized over the entire planning horizon. For sampled scenarios, a rolling horizon is employed to assess feasibility, where demand forecasts for future periods are used to determine the optimal movement of goods in the current period. Algorithm 2 is the general structure of this framework. By evaluating this algorithm for N different scenarios, an overall overview of feasibility could be obtained.

Algorithm 2 Rolling Horizon Multi-cut Benders Decomposition Algorithm

```
Set fixedDay = 0, result = −∞, feasible ← true.
while while fixedDay ≤ T ∧ feasible do
    Set iter = 1.
    if Solution exists for previous iteration then
        Provide MILP solver with warm start
    end if
    while There exists a subproblem that is not feasible do
        Solve the master problem (MP) with all feasibility cuts from previous iterations.
        if MP is not feasible then
            feasible ← false
            return
        end if
        Let the optimal objective value be φ^iter and the first-stage solution be X^iter.
        if φ^iter > result then
            result ← φ^iter
        end if
        Solve the subproblems with x̄ = X^iter.
        For all subproblems, check if the subproblem is feasible.
        if Subproblem is infeasible then
            Compute the dual rays of the subproblem α^iter_{f,u,t}, β^iter_{k,p,t}, γ^iter_{k,p,t}, δ^iter_{k,p,t} and η^iter_{f,t}
            Add the feasibility cut (34) or (35) to the master problem.
        end if
        iter ← iter + 1
    end while
    OutputInner: The optimal solution X^iter.
    fixedDay ← fixedDay + 1.
    Fix X^iter for day fixedDay in the master problem.
end while
Output: result, feasible
```

To speed up this algorithm and increase feasibility, a few addition are introduced. First, since only the current period needs to be optimized to integrality in each iteration, the integrality constraints of the future periods can be relaxed. Demand realization of the current period may shift the specific shipments in future periods, which make the specific schedule for future periods less important. Let H_{SMCBD} be the days of the look-ahead integrality of the current iteration. The value of H_{SMCBD} is determined in Section 6.2.2.

A second modification concerns the demand forecast for future periods. In the deterministic setting, demand for each commodity is forecast as the mean of the provided demand distributions. However, for commodities with high variability, stock-outs may occur more frequently due to the low margin of error in the per-period forecast. To address this, a safety margin θ is introduced as a multiple of the standard deviation added to the forecasted demand. The effect of varying θ on solution feasibility is examined experimentally in Section 7.2.

6 Computational Setup

This chapter describes the computational setup used to evaluate the performance of the proposed approaches. We introduce the problem instances, along with the parameter settings applied for the case of stochastic and deterministic demand. The results of these experiments are subsequently presented in Chapter 7.

6.1 Problem Instances

To evaluate the computational performance of the discussed methods, several instances of varying sizes are considered, which differ in the number of OUs and FSCs. This collection of instances consists of a *Small*, a *Normal*, and a *Large* instance. These three instances are considered as the *problem size* instances. Additionally, we construct two dispersed instances by adding FSCs to the Normal instance. In the first dispersed instance, one additional FSC is introduced at a location between the existing FSCs. This FSC takes over several distribution arcs from the existing FSCs such that each FSC serves an approximately equal number of OUs. In the second dispersed instance, two additional FSCs are added following the same principle. The instance characteristics are summarized in Table 1.

Table 1: Number of variables for each instance.

	Nodes			Variables per T		
Instance	MSCs	FSCs	OUs	Integer	Continuous	Total
Small	1	1	5	96	31	127
Normal	1	2	11	208	65	263
Dispersed 1	1	3	11	224	81	305
Dispersed 2	1	4	11	240	97	337
Large	1	4	20	384	124	508

The Normal instance, illustrated in Figure 1, serves as the base case and was provided by TNO. The Small instance is a reduced version of the Normal instance, while the Large instance is an expanded version with additional FSCs and OUs. The dispersed instances add FSCs exclusively for load balancing purposes. These network structures are illustrated in Appendix B, and corresponding capacities and daily deterministic demand per OU and commodity are presented in Appendix C. We evaluate all instances for planning horizons $T \in \{5, 10, \ldots, 100\}$. The number of variables scales linearly with T, such that longer planning horizons increase the problem size proportionally.

6.2 General parameters

This section presents the general parameter settings. General parameters apply to both the deterministic and stochastic setting, while setting-specific parameters are described separately.

Each CCL type is characterized by a fixed composition of commodities, defined by the parameter $b_{u,p}$, which specifies the quantity of commodity $p \in P$ contained in a combination of CCL and OU type $u \in U$. The CCL composition is independent of the OU destination. We consider four CCL types. A fourth type is introduced based on observations in the demand data. As stock-out situations occur most frequently for fuel, additional attention is given to fuel-related configuration options. The compositions of the four CCL types considered, are shown in Table 2.

Table 2: Overview of composition of CCL type per commodity

CCL Type	Food & Water (kg)	Fuel (kg)	Ammunition (kg)
Type 1	3000	3000	4000
Type 2	1000	4000	5000
Type 3	0	2000	8000
Type 4	2000	8000	0

The storage capacity c_f of each FSC varies per FSC $f \in F$ and is expressed in terms of CCLs. Furthermore, storage capacity $c_{k,p}$ of each OU varies for each individual OU $k \in K$ and per commodity $p \in P$, and is expressed in kilograms. The initial inventory levels $q_{f,u}$ vary per FSC $f \in F$ and type $u \in U$. Initial inventories for the OUs $k \in K$ are full, so at their maximum inventory. These parameter tables are summarized in Appendix C.

6.2.1 Deterministic Parameters

In the deterministic setting, the daily demand $d_{k,p,t}$ is assumed to be constant over time and known for each OU $k \in K$, commodity $p \in P$ and time period $t \in T$. Demand only varies per individual OU and commodity. The complete demand specifications are provided in Appendix C.

6.2.2 Stochastic Parameters

To model stochastic demand, let $\bar{d}_{k,p,t}$ denote the nominal (average) daily demand of commodity p at OU k during period t, where $\bar{d}_{k,p,t}$ corresponds to the deterministic demand values described in Section 6.2.1. The realized demand is given by the following equation:

$$d^{\xi}_{k,p,t} = \bar{d}_{k,p,t} \cdot \zeta^{\xi}_{k,p,t}, \tag{42}$$

for commodities food & water, and ammunition. Notably, for the fuel commodity we divide the value obtained by equation 42 by T, where $\zeta^{\xi}_{k,p,t}$ is a scenario-dependent multiplicative factor drawn from a commodity-specific probability distribution:

$$\zeta^{\xi}_{k,p,t} = \begin{cases} \mathcal{U}(0.8, 1.2), & p = \text{food \& water} \\ \mathcal{B}(25, 0.4), & p = \text{fuel} \\ \mathcal{T}(0.2, 0.8, 2.0), & p = \text{ammunition} \end{cases} \quad , \forall \xi \in S \tag{43}$$

These distributions are used exclusively for scenario generation. Each scenario $\xi \in S$ corresponds to one deterministic realization of demand over all commodities, OUs, and the complete time horizon.

Throughout the experiments, we use $N = 1000$ scenarios in all solution methods.

For the rolling horizon SAA approach, we fix $N_{SAA} = 20$ in-sample scenarios per epoch. We run preliminary experiments, to find the most suitable look-ahead window H_{SAA}. To improve computational efficiency, bounds $[Z_{\min}, Z_{\max}]$ are defined to narrow the search interval for the optimal solution. If the optimal solution coincides with either bound, the interval is shifted by 20 units in the corresponding direction and the problem is re-solved. This process repeats until the optimal solution lies strictly within the interval. The initial bounds for each instance are provided in Appendix C.

For the SMCBD, we use H_{SMCBD} as the rolling horizon look-ahead window. This parameter value is fixed to $H_{SMCBD} = 5$ throughout all experiments.

7 Computational Results

This section presents the results of the computational experiments. Section 7.1 first reports the results for the deterministic setting, covering both the problem size instances and the dispersed instances. Section 7.2 then follows the same structure for the stochastic setting. Subsequently, Section 7.3 examines the sensitivity of the results to variations in the service level parameter α. Finally, Section 7.4 presents the long-term results for the deterministic setting under an extended time horizon.

All methods are implemented in Java and solved using Gurobi version 12.0.0 with a time limit of two hours (7200 seconds). The computational experiments are conducted on a MacBook Air with an Apple M3 chip and 8 GB of RAM.

7.1 Results - Deterministic setting

This section reports and discusses the results for the deterministic setting at $T = 10$. Each solution method has a time limit of 7200 seconds (2 hours).

Table 3: Fleet size and runtimes (s) to optimality for each method and $T = 10$.

Instance	Optimal fleet size	B&C	MCBD	AMCBD
Small	56	0.05	0.17	2.82
Normal	101	0.25	3.80	63.68
Large	178	5.58	15.48	7200*

* Time limit of 7200 seconds reached; optimality could not be confirmed.

Table 3 reports the minimum fleet size and corresponding runtime (in seconds) for each instance and solution method under deterministic demand at $T = 10$. Since both B&C and MCBD solve the MILP to optimality, we observe an identical fleet size across these approaches. Both methods certify optimality for all instances within the imposed time limit, with B&C consistently outperforming MCBD in terms of computational efficiency. Specifically, MCBD requires approximately 3, 15, and 3 times the runtime of B&C for the Small, Normal, and Large instances, respectively.

A comparison between the objective values shows the required fleet size reduces by 45% when the instance size decreases from Normal to Small; the optimal solution decreases from 101 to 56. On the other hand, the Large instance requires a fleet capacity of 178 trucks, representing an increase of 76% compared to the Normal instance.

These results highlight the substantial difference in computational performance between the two MILP formulations. Specifically, in contrast to the other solution methods, AMCBD reaches the time limit for the Large instance and is therefore unable to certify optimality in this case.

The MCBD algorithm exhibits rapid convergence with respect to the fleet size, yet requires considerably more iterations to certify a feasible shipment and inventory plan, which partially accounts for its longer runtime relative to B&C.

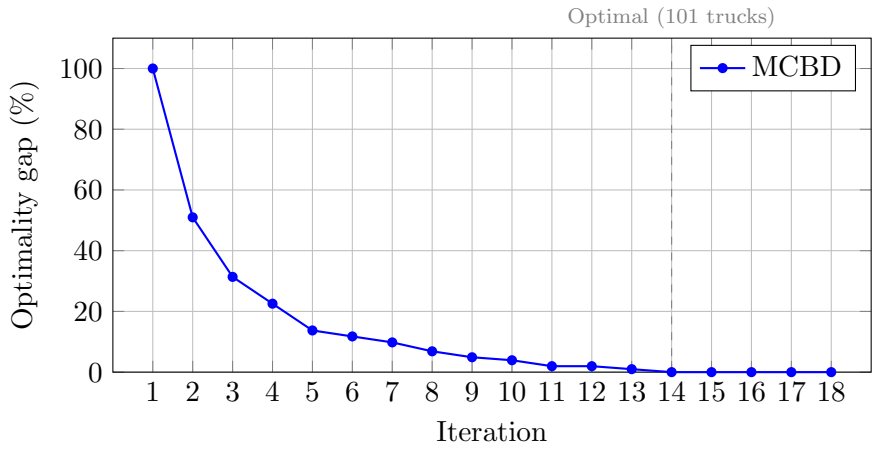

Figure 3: Convergence of the MCBD method: optimality gap as a function of the iteration.

Figure 3 illustrates the convergence behavior of the MCBD method in terms of the optimality gap as a function of the number of iterations. The gap decreases rapidly in the early iterations and reaches zero at iteration 14, well before the algorithm terminates. However, this does not imply that a feasible shipment plan has been found at this point. When the gap first hits zero, the subproblems remain infeasible with respect to the shipment and inventory plan induced by the current flow solution of the master problem. The algorithm therefore continues to generate and add feasibility cuts, re-solving the master problem in each subsequent iteration, even though the objective value no longer improves. This explains the flat region at 0% gap from iterations 14 to 18, which accounts for a substantial proportion of the total runtime. Only at the final iteration is a feasible subproblem obtained, certifying that the identified fleet size is both optimal and feasible. This behavior reveals a critical characteristic of MCBD: the bottleneck lies not in identifying the optimal objective value, but in certifying the existence of a feasible shipment plan.

Having examined computational performance, we now turn to the structure of the optimal solution. For practical insights, we further examine how trucks are distributed across the warehouses. The absolute and relative distribution of trucks in the optimal solution is analyzed for each instance.

Table 4: Absolute and relative optimal truck allocation for the problem size instances.

	Absolute		Relative	
Instance	MSC	FSCs	MSC	FSCs
Small	27	29	48%	52%
Normal	43	58	43%	57%
Large	71	107	40%	60%

Table 4 presents the absolute and relative truck allocation per warehouse for the problem size instances in the optimal solution. A notable pattern is that as the instance size increases, a larger proportion of trucks is allocated to the FSCs rather than to the MSC. Specifically, the share of trucks stationed at the FSCs increases from 52% for the Small instance, to 57% for the Normal instance, and to 60% for the Large instance. This can be attributed to the composition of the instances, as reported in Table 1. In particular, the number of OUs as a fraction of total locations increases with instance size, from 71% for the Small instance, to 79% for the Normal instance, and to 80% for the Large instance.

7.1.1 Results - Dispersed Deterministic Setting

For the deterministic setting, the results for the base case and the dispersed instances are presented in Table 5 for $T = 10$. The results are computed with the B&C solution method, which was identified as the preferred method in Section 7.1.

Table 5: Fleet size and runtime (s) for the dispersed instances at $T = 10$.

Instance	Fleet size	Runtime (s)
Normal	101	0.249
Dispersed 1	105	0.988
Dispersed 2	106	0.275

Table 5 presents the optimal minimum fleet size and runtime for each dispersed instance. Notably, the minimum fleet sizes for the dispersed instances are 4–5% higher than for the Normal instance. This is contrary to the expectation that additional FSCs improve efficiency and reduce the required fleet size by better geographic dispersion. All instances are solved within one second. To investigate this increase in optimal fleet size when introducing dispersion, we also look at the absolute and relative optimal truck allocations.

Table 6: Absolute and relative optimal truck allocation for the problem size instances.

	Absolute		Relative	
Instance	MSC	FSCs	MSC	FSCs
Normal	43	58	43%	57%
Dispersed 1	47	58	45%	55%
Dispersed 2	48	58	45%	55%

Table 6 presents the absolute and relative fleet allocations for each dispersed instance in the optimal solution. The results indicate that while the FSC fleet remains constant at 58 trucks across all instances, the MSC absorbs the entire increase in fleet size. The introduction of additional FSCs thus increases the number of trucks required at the MSC. This can be attributed to the fact that more FSCs imply more resupply flows from the MSC to each FSC. In contrast, demand at the OUs remains unchanged, resulting in a stable truck allocation at the FSCs across all instances.

7.2 Results - Stochastic setting

This section presents the results of the solution methods under stochastic demand and a time horizon of $T = 10$ days. A service level of $\alpha = 95\%$ is imposed, which implies permitting a stock-out in at most 5% of the simulated scenarios. Prior to presenting the results, preliminary analyses of the parameter settings for both the SAA and the SMCBD are conducted.

To select an appropriate value for the look-ahead window H_{SAA} for the SAA, we evaluate configurations $H_{SAA} \in \{2, 3, 4, 5\}$ on the Normal instance and initial fleet size bounds $Z_{\min} = 150$ and $Z_{\max} = 200$. Sample size parameters N and N_{SAA} are fixed as specified in Section 6.2.2.

Table 7: Objective of look-ahead window configurations for SAA.

H	Fleet size	Runtime (s)
2	162	937.39
3	154	5515.80
4	149	12099.72
5	148	16721.93

Table 7 presents the minimum fleet size and runtime for look-ahead windows $H_{SAA} \in \{2, 3, 4, 5\}$ for service level $\alpha = 95\%$. The results clearly indicate that a larger look-ahead window yields solutions with a lower optimal fleet size for the given service level, due to more forward information. However, this comes at the cost of a substantially increased computational burden. Given this trade-off between solution quality and runtime, the configuration $H_{SAA} = 3$ is selected for further analysis.

Secondly, an appropriate value of θ, the safety coefficient, is determined. We evaluate the service level and minimum fleet size for $\theta \in \{0, 0.5, \ldots, 2.5, 3\}$ when solving the Normal instance. For all other instances the parameter settings can be found in Appendix F. Furthermore, the parameters H_{SMCBD} and N remain as defined in Section 6.2.2.

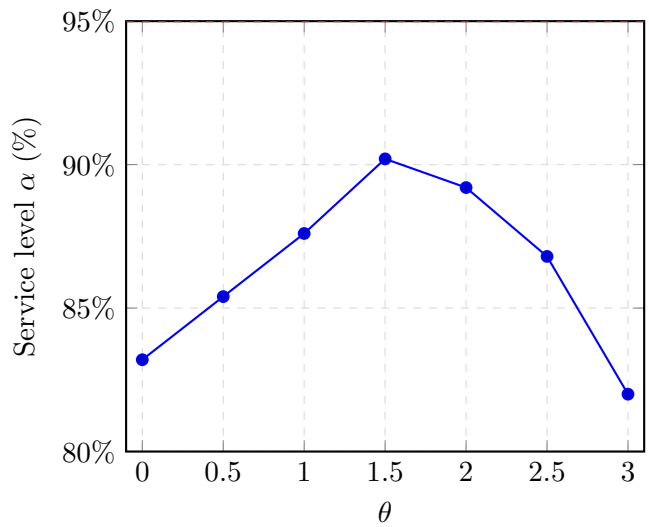

Figure 4: Service level α as a function of the safety coefficient θ.

Figure 4 illustrates the achieved service level α as a function of θ, evaluated on the Normal instance. The service level peaks at $\alpha = 90.2\%$ for $\theta = 1.5$, beyond which it decreases monotonically. Notably, a service level of 95% cannot be achieved for any value of θ considered, suggesting that adjusting the expected demand with the safety coefficient alone is insufficient to meet the target service level on the Normal instance. This indicates that the cuts we have considered in the SMCBD are not conservative enough to cope with the demand uncertainties.

Having established the appropriate parameter settings, the results for each instance and solution method are presented in Table 8.

Table 8: Results for each problem size instance in the stochastic setting.

	SMCBD		SAABS		SAABBS	
Instance	Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)
Small	82	1952.67	70	1502.26	72	1888.06
Normal	-	-	154	5515.80	158	5790.98
Large	-	-	320	7601.63	322	8373.87

* – indicates that the required service level of 95% was not attained.

Table 8 presents the minimum fleet size and runtime for each problem size instance under each solution method. Notably, SMCBD is unable to find a feasible solution satisfying $\alpha \geq 95\%$ for the Normal and Large instances, whereas it does certify a solution for the Small instance.

For the Small instance, SAABS yields the best solution with a minimum fleet size of 70 trucks, followed by SAABBS with 72 trucks. SMCBD performs worst among the three methods, requiring a minimum fleet size of 82 trucks.

For the Normal and Large instances, SAABS consistently yields solutions with a smaller fleet size than SAABBS. This is consistent with expectations, as BBS is a more conservative extension of BS, and therefore tends to require a larger fleet size.

In the following section, the dispersed instances under stochastic demand are examined.

7.2.1 Results – Dispersed Stochastic Setting

For the dispersed instances, the stochastic results are compared against those of the Normal instance. These findings reveal several notable patterns.

Table 9: Results for the dispersed instances in the stochastic demand setting.

	SMCBD		SAABS		SAABBS	
Instance	Obj.	Runtime (s)	Obj.	Runtime (s)	Obj.	Runtime (s)
Normal	–	–	154	5515.80	158	5790.98
Dispersed 1	–	–	170	4167.59	174	4253.14
Dispersed 2	–	–	176	3844.84	179	4054.55

* – indicates that the required service level of 95% was not attained.

Table 9 reports the objective values and runtimes for all solution methods. The following findings emerge.

As observed in the deterministic setting, the dispersed instances require larger fleet sizes than the Normal instance. Specifically, Dispersed 1 requires at least 170 trucks, representing a 10% increase relative to the Normal instance (154 trucks), while Dispersed 2 requires 176 trucks, corresponding to an increase of 14%. This is consistent with the findings reported in Section 7.1.1, where the same pattern was attributed to the increased number of resupply flows from the MSC to the additional FSCs.

Furthermore, despite requiring larger fleet sizes, the dispersed instances are solved considerably faster than the Normal instance. Specifically, Dispersed 1 is solved in around 4200 seconds and Dispersed 2 in around 3800 seconds, compared to approximately 5500 seconds for the Normal instance. This suggests that a greater dispersion of operating units across FSCs simplifies the underlying optimization problem.

Finally, consistent with earlier observations, SAABBS requires marginally more trucks and slightly longer runtimes than SAABS across all instances, though the differences remain small.

The SMCBD approach again fails to produce results, reinforcing the conclusion that the SAA-based methods are the more scalable choice to solve the stochastic resupply problem across diverse instance types.

Having examined the results under a fixed service level of 95%, the following section investigates the sensitivity of the fleet size to variations in the service level parameter α.

7.3 Results - Variations of the Service Level Parameter

Since the service level $\alpha = 95\%$ could not be achieved for the Benders decomposition across all instances, this section compares between the Benders decomposition and the rolling horizon approaches under lower service level requirements.

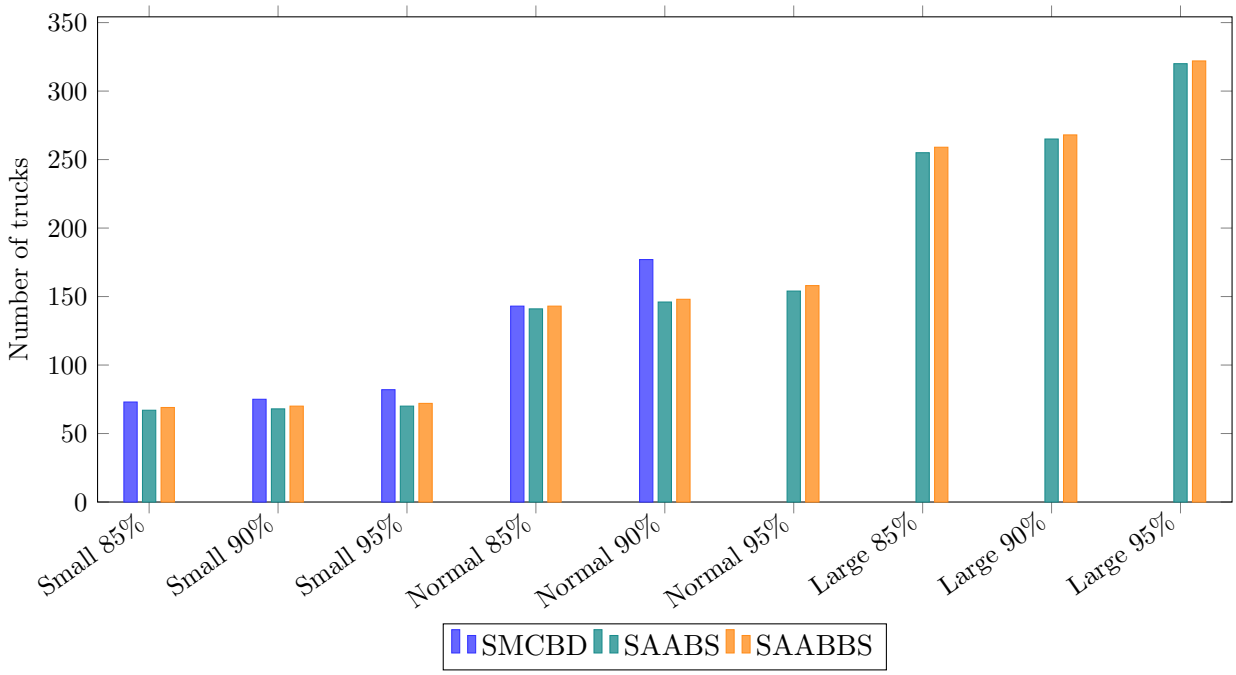

Figure 5: Minimum fleet size per solution method, instance, and service level α.

Figure 5 presents the results for all three solution methods across the three problem size instances and service levels, obtained by simulating $N = 1000$ scenarios with varying safety coefficients θ depending on the instance.

For the Small instance, the required service level is achieved by all solution methods. However, SMCBD consistently yields larger fleet sizes than SAABS and SAABBS, requiring 82 trucks at $\alpha = 95\%$, 75 trucks at $\alpha = 90\%$, and 73 trucks at $\alpha = 85\%$. For SAABS, a minimum fleet size of 70 trucks is required at $\alpha = 95\%$, decreasing to 68 trucks at $\alpha = 90\%$ and to 67 trucks at $\alpha = 85\%$.

For the Normal instance, a service level of $\alpha \leq 90\%$ enables all methods, including SMCBD, to find a feasible solution. At $\alpha = 85\%$, SMCBD and SAABBS both require a minimum fleet size of 143 trucks, while SAABS yields a slightly smaller fleet size of 141 trucks. For $\alpha = 90\%$, the results diverge more substantially: SMCBD requires 177 trucks, whereas SAABS and SAABBS require 146 and 148 trucks, respectively.

The Large instance cannot be solved to feasibility by SMCBD at any of the evaluated service levels. In contrast, SAABS and SAABBS remain able to find solutions, though the service level requirement has a considerable influence on the resulting fleet size. Reducing the service level from

$\alpha = 95\%$ to $\alpha = 90\%$ yields a substantial decrease of approximately 17% in the minimum fleet size, from 320 to 265 trucks for SAABS and from 322 to 268 trucks for SAABBS. A further reduction to $\alpha = 85\%$ yields additional decreases to 255 and 259 trucks for SAABS and SAABBS, respectively.

Across all instances, the sensitivity of the minimum fleet size to the service level requirement increases with instance size, underscoring the growing trade-off between service level and fleet size as the problem scales. Furthermore, SMCBD consistently yields larger fleet sizes than SAABS and SAABBS where feasible, and fails entirely for the Large instance, confirming the superior scalability of the SAA-based methods.

7.4 Results - Extended operational period

This section presents the results of the proposed solution methods across a range of operational periods extending beyond the ten-day base case considered thus far. Results are reported for the deterministic setting only, as the stochastic methods were unable to achieve the required 95% service level within a reasonable runtime. This can be explained by the increased likelihood of a stock-out occurring over longer time horizons, which the stochastic methods are unable to adequately handle.

The dispersed instances are excluded from these experiments, as varying the operational period does not provide additional insight beyond what is already captured by these problem size instances. The dispersed configurations differ primarily in their location distribution rather than in how fleet capacities scale with the operational period.

Two planning horizons are highlighted for each problem size instance: $T = 10$ and $T = 60$, as they represent two practically relevant cases: $T = 10$ corresponds to the baseline case, while $T = 60$ represents two full months and allows us to assess the scalability of the proposed methods over a longer planning horizon to find long-term solutions.

Table 10 presents the results for $T = 60$. The results for $T = 10$ can be found in Table 3.

Table 10: Runtimes (s) till optimality for each method and $T = 60$.

Instance	Optimal fleet size	B&C	MCBD	AMCBD
Small	82	1.07	20.95	7200*
Normal	148	26.27	423.76	7200*
Large	263	182.85	2143.81	7200*

* Time limit of 7200 seconds reached; optimality could not be confirmed.

Table 10 presents the minimum fleet size for $T = 60$. Extending the planning horizon from $T = 10$ to $T = 60$ days results in a substantial increase in the required fleet size, as illustrated by the Normal instance, where the minimum fleet size grows from 101 to 148 trucks. Notably, AMCBD, which could be solved within 60 seconds for the Normal instance at $T = 10$, fails to certify optimality within a 2-hour time limit at $T = 60$, highlighting a significant increase in computational complexity. A similar trend is observed for B&C, where the runtime increases by approximately a factor of 100 for the Normal instance. Taken together, these results suggest that AMCBD is unlikely to remain tractable for larger instances and longer planning horizons, particularly given its already considerable computational effort in the base case.

To further investigate the runtime of each solution method, the following figure illustrates how the runtime scales with the number of variables in the model for each solution method.

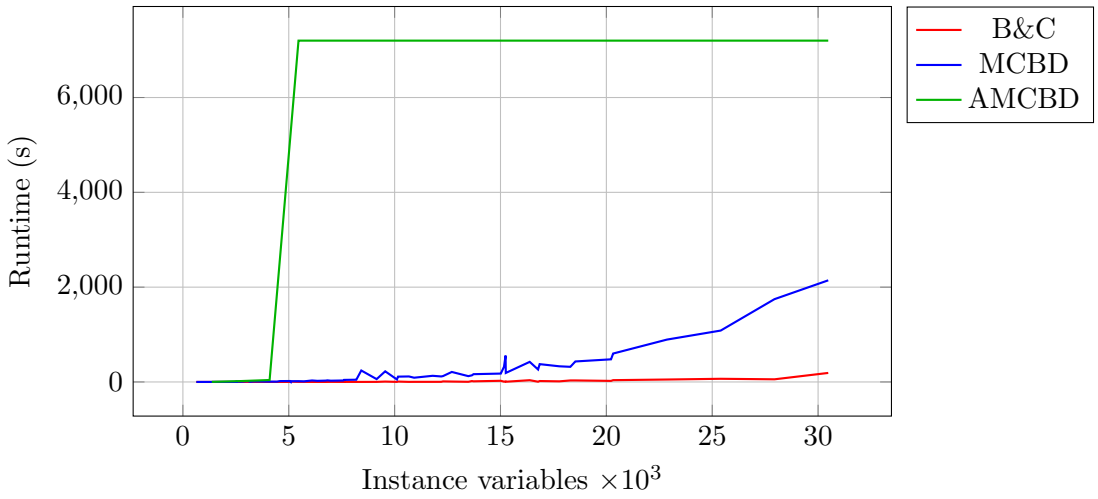

Figure 6: Runtime as a function of the number of instance variables for each solution method.

Figure 6 illustrates the runtime (in seconds) of the three solution methods as a function of the number of variables in the model, subject to a time limit of 7200 seconds. B&C consistently achieves the lowest runtime as the problem size increases. MCBD also certifies optimality for all instances and time horizons considered, albeit with considerably longer computation times. As reported in Table 10, B&C solves the Large instance more than ten times faster than MCBD. Moreover, the computational disadvantage of MCBD relative to B&C grows with instance size, which is contrary to the general expectation that Benders decomposition outperforms branch-and-cut for large-scale instances. This behavior can be attributed to the large number of iterations required to certify feasibility of the shipment plan once the optimal fleet size has already been identified, as discussed in Section 7.1.

In contrast to the other solution methods, AMCBD reaches the time limit rapidly as the number of variables grows, and is therefore unable to certify optimality for larger instances. This indicates that the scalability of AMCBD deteriorates significantly with increasing operational period and instance size, highlighting the inferiority of the alternative formulation for large-scale problems.

Given that B&C consistently certifies optimality across all instance sizes and operational periods within the imposed time limit, and that it is the most computationally efficient method among those considered, it is selected as the preferred solution method for the deterministic setting. The optimal solutions for each instance size and time horizon are presented in Figure 7.

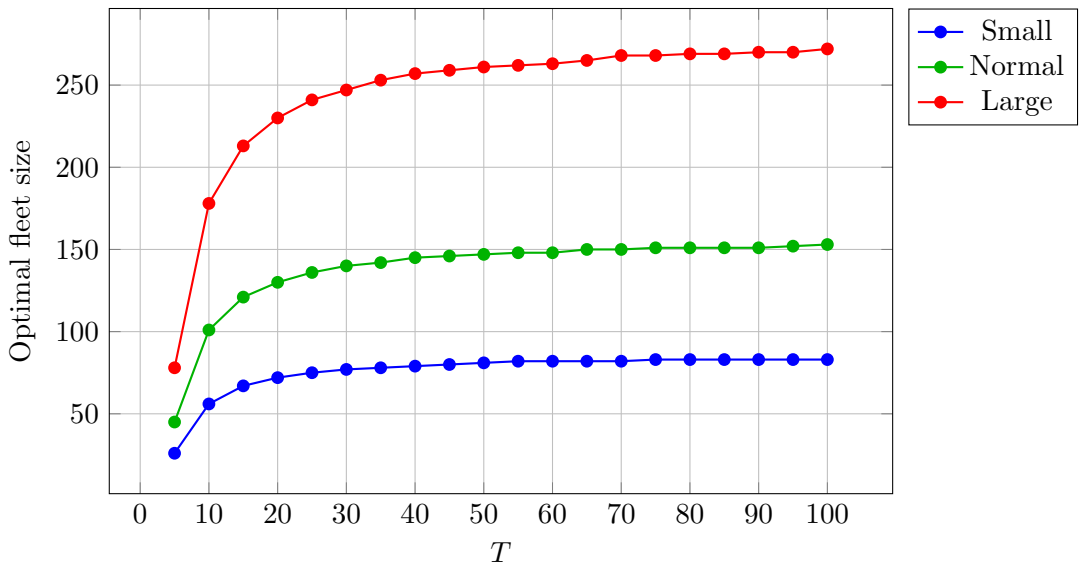

Figure 7: Optimal fleet size of the problem size instances as a function of the time horizon T for deterministic demand.

Figure 7 illustrates the optimal minimum fleet size as a function of the time horizon T for each of the problem size instances. It is evident for all of these instances that the optimal fleet size increases steeply at small values of T and gradually stabilizes as the time horizon grows. Furthermore, as the instance size increases, this stabilization occurs at larger values of T.

For the Small instance and the base case of $T = 10$, the optimal fleet size is 56 trucks. Extending the time horizon to $T = 60$ yields an optimal fleet size of 82 trucks, representing a 46% increase relative to the base case. The optimal fleet size increases only marginally beyond this point, reaching 83 trucks at $T = 100$.

For the Normal instance, at $T = 10$, the optimal fleet size is 101 trucks. Extending the time horizon to $T = 60$ days results in an optimal fleet size of 148 trucks, a 47% increase. A further extension to $T = 100$ days yields a fleet size of 153 trucks, indicating that stabilization is nearly reached within the evaluated range.

For the Large instance, stabilization is the slowest among all instances, consistent with its greater size. Indeed, the optimal fleet size continues to increase across all evaluated values of T. At $T = 10$ days, the minimum required fleet size is 178 trucks. This increases by 48% to 263 trucks at $T = 60$, and by 53% to 272 trucks at $T = 100$. However, the long-run optimal fleet size cannot be determined within the range of time horizons considered in this study.

Overall, a comparison of the optimal fleet sizes across time horizons $T = 10$ and $T = 60$ reveals substantial differences in the optimal solution. As reported in Tables 3 and 10, and illustrated in Figure 7, the optimal fleet size at $T = 60$ is, on average, 47% larger than at the base case of $T = 10$. This suggests that relying solely on the base case leads to a considerable underestimation of the minimum fleet size required in the long-term.

8 Conclusion

This research studies a multi-period fleet sizing problem for a military resupply network. The network consists of a main supply center (MSC), multiple forward supply centers (FSCs), and operating units (OUs). The objective is to determine the minimum number of trucks required to guarantee a feasible shipment and allocation plan over a finite planning horizon, while satisfying transportation and inventory constraints.

Returning to the research questions posed in Chapter 1, the results provide the following answers. First, a minimum of 101 trucks is required to prevent stock-outs under deterministic demand over a ten-day planning horizon, increasing to 154 trucks when stochastic demand is considered at a 95% service level — an increase of over 50%. Second, the required fleet size does not grow indefinitely with the planning horizon; under deterministic demand, it stabilizes at 153 trucks as the horizon extends, providing the Royal Netherlands Army (RNA) with a robust and concrete long-term planning target independent of exact mission duration.

In addition to the baseline scenario, varying instance sizes are examined to provide insights into military logistics planning, given the substantial practical relevance of fleet sizing decisions at different scales. To this end, different sizes of the given network configuration are considered. As expected, larger instances require larger fleet sizes. A notable finding is that SMCBD fails to produce results in the stochastic setting at a 95% service level for any instance except the Small instance. To further investigate this limitation, multiple service levels are examined. These results confirm that a higher service level requires a larger fleet size across all methods. Crucially, the SAA-based approach successfully solves all instance sizes up to $\alpha = 95\%$, whereas SMCBD only achieves this service requirement for a smaller version of the base case instance. The standard network configuration under SMCBD could only be solved up to $\alpha = 90\%$, and the large instance could not be solved at all. This demonstrates that the SAA approach consistently outperforms Benders decomposition in terms of scalability, runtime, and problem coverage, making SAA-based methods the more practical choice for large-scale military logistics problems.

Beyond instance size, different FSC configurations were also examined, by adding FSCs and introducing dispersion to the instances in the deterministic setting. These results indicate that approximately 4–5% more trucks are required compared to the baseline instance in the deterministic setting, and 10–14% in the stochastic setting. Furthermore, the distribution of trucks across the warehouses shifts noticeably: when additional FSCs are introduced in the dispersed cases, the extra required trucks are primarily allocated to the MSC.

To contextualise the practical scale of the findings, a comparison can be made with the Scania Gryphus, the most widely used truck within the RNA. Given that the RNA currently operates approximately 2,400 Scania Gryphus trucks Ministerie van Defensie (2024), the mission sizes analysed in this study represent only a small fraction of the total available fleet, confirming that the scenarios examined are representative of realistic operational scales and well within the RNA's current logistical capacity.

While these findings provide actionable guidance for the RNA, several limitations suggest directions for future research. Two directions are particularly promising. First, it would be valuable to investigate the impact of incorporating different truck types with varying capacities. As the RNA is expected to introduce several new types of trucks in the near future Ministerie van Defensie

(2025), understanding how mixed fleets affect logistics performance is increasingly relevant. Larger trucks may be better suited for high-volume transport between the MSC and FSCs, while smaller and more agile vehicles may prove more effective for last-mile distribution to OUs. Studying the optimal combination and allocation of truck types could provide meaningful improvements in both efficiency and flexibility.

Second, future research could examine how logistics systems perform under dynamic and shifting battlefield conditions. In real-world conflicts, the operational environment rarely remains static. For example, in the ongoing conflict in Ukraine, the battlefield continuously evolves as operations progress in different directions. As a result, the demand of certain OUs may change substantially over time, and some locations may become inaccessible or strategically irrelevant. In certain situations, operational areas may even be lost or destroyed, eliminating the need for further supply. Future research could therefore investigate how logistics systems can adapt to these dynamic conditions and shifting operational priorities, recognizing that in military strategy it can sometimes be necessary to *lose a battle in order to win the war*.

References

Bartl, D. (2007). Farkas' lemma, other theorems of the alternative, and linear programming in infinite-dimensional spaces: a purely linear-algebraic approach. *Linear and Multilinear Algebra*, 55(4):327–353. [Accessed: 12-03-2026].

Beheshti Asl, N. and MirHassani, S. A. (2018). Accelerating Benders decomposition: multiple cuts via multiple solutions. *Journal of Combinatorial Optimization*, 37:806–826. [Accessed: 12-03-2026].

Bidhandi, H. M., Yusuff, R. M., Megat Ahmad, M. M. H., and Abu Bakar, M. R. (2009). Development of a new approach for deterministic supply chain network design. *European Journal of Operational Research*, 198:121–128. [Accessed: 10-03-2026].

Cardoso, S. R., Barbosa-Póvoa, A. P., and Relvas, S. (2014). An accelerated Benders decomposition algorithm for sustainable supply chain network design under uncertainty. *Transportation Research Part E: Logistics and Transportation Review*, 67:14–38. [Accessed: 05-03-2026].

Chuong, T. D., Liu, C., and Yu, X. (2025). Decomposition for large-scale optimization problems: An overview. *Artificial Intelligence Science and Engineering*. [Accessed: 20-02-2026].

Mancilla, C. and Storer, R. H. (2012). A sample average approximation approach to stochastic appointment sequencing and scheduling. *IIE Transactions*, 44:655–670. [Accessed: 10-03-2026].

Ministerie van Defensie (2024). Scania Gryphus-transportvoertuig. [Accessed: 10-03-2026].

Ministerie van Defensie (2025). 785 nieuwe operationele wielvoertuigen voor Defensie. [Accessed: 10-03-2026].

Singh, V., Khanzadeh, M., Davis, V., Rush, H., Rossi, E., Shrader, J., and Lio, P. (2024). Bayesian binary search. *arXiv preprint arXiv:2410.01771*. [Accessed: 20-02-2026].

Wolfram Research, Inc. (2024). Beta distribution. [Accessed: 16-02-2026].

You, F. and Grossmann, I. E. (2013). Multicut Benders decomposition algorithm for process supply chain planning under uncertainty. *Annals of Operations Research*, 210(1):191–211. [Accessed: 05-03-2026].

Appendix

A Multi-cut Benders Decomposition method

In this method, a scenario tree is adopted, analogously to Section 5.1. This means that the truck flows on day $t = 1$ are identical across all scenarios, corresponding to the root node of the scenario tree. For $t > 1$, the scenarios branch independently. For demand scenario $\xi \in S$, let $\xi(t)$ denote the period of scenario ξ at time $t \in T$, such that $\xi := \{\xi(1), \ldots, \xi(T)\}$. By constructing, it is possible for scenario's to have the same demand history up till an certain point, from where the scenario's branch. This branching of scenario can create lots of nodes in the tree, but is the mechanism in which perfect-foresight is addressed. Additionally, it means that a scenario can be defined by its leaf nodes at the end of the planning horizon, from where the shortest path to the root node is its scenario. Let $\upsilon_{\xi(t)} \in \{0, 1\}$ be a binary variable indicating whether period t of scenario ξ is infeasible. Let $\kappa_\xi \in \{0, 1\}$ be a binary variable indicating whether at least one period in scenario ξ is infeasible. This yields the following Benders master problem:

$$\min \quad \sum_{i \in L \setminus K} z_i \tag{44}$$

$$\text{s.t.} \quad z_i \geq \sum_{j:(i,j) \in A} \sum_{u \in U} (x_{i,j,u,\xi(t)} - x_{i,j,u,\xi(t-1)}) \qquad \forall\, i \in L \setminus K,\ t \in T, \xi \in S \tag{45}$$

$$x_{i,j,u,\xi(t-1)} \leq x_{i,j,u,\xi(t)} \qquad \forall\, (i,j) \in A,\ u \in U,\ t \in T, \xi \in S \tag{46}$$

$$\upsilon_{\xi(t)} \leq \kappa_\xi \qquad \forall\, t \in T,\ \xi \in S \tag{47}$$

$$\sum_{\xi \in S} \kappa_\xi \leq (1 - \alpha)|S| \tag{48}$$

$$x_{i,j,u,\xi(t)} \in \mathbb{Z}_+ \qquad \forall\, (i,j) \in A,\ u \in U,\ t \in T, \xi \in S \tag{49}$$

$$z_i \in \mathbb{Z}_+ \qquad \forall\, i \in L \setminus K. \tag{50}$$

The objective is to minimize the total fleet size. Constraint (45) enforces that the trucks used at a warehouse are consistent. Constraint (46) strengthens the formulation by explicitly enforcing continuity between periods. Constraint (47) checks scenario feasibility, which controls the service level in constraint (48). Feasibility of a scenario is enforced with the feasibility cuts of the SPs. Through the variable $\upsilon_{\xi(t)}$, a feasibility cut is allowed to be violated if $\upsilon_{\xi(t)} = 1$. This allows us to enforce a service level viability (no stock-out) in the master problem. Let M be a large enough number such that the feasibility cuts are always feasible. For every infeasible SP, given its dual rays, the feasibility cuts are constructed as follows:

$$\begin{aligned} \sum_{u \in U} \alpha^{iter}_{f,u,\xi(t)} (q_{f,u} + \sum_{i:(i,f) \in A} x_{i,f,u,\xi(t)} - \sum_{j:(f,j) \in A} x_{f,j,u,\xi(t)}) + \eta^{iter}_{f,t} c_f \\ \leq M \upsilon_{\xi(t)}, \quad \forall\, f \in F,\ t \in T,\ \xi \in S, \end{aligned} \tag{51}$$

$$\begin{aligned} \beta^{iter}_{k,p,\xi(t)} (c_{k,p} + \sum_{i \in L \setminus K} \sum_{u \in U_k} x_{i,k,u,\xi(t)} b_{u,p} - \sum_{m=1}^{\xi} (t) d_{k,p,\xi(m)}) + \delta^{iter}_{k,p,\xi(t)} c_{k,p} - \gamma^{iter}_{k,p,\xi(t)} d_{k,p,\xi(t)} \\ \leq M \upsilon_{\xi(t)}, \quad \forall\, k \in K, p \in P,\ t \in T,\ \xi \in S. \end{aligned} \tag{52}$$

The SPs remain largely identical to those introduced in Section 4.2.2. The only modification is that SPs are now generated for each $\xi(t)$ rather than for each $t \in T$, which substantially increases the total number of SPs. However, since each SP is an LP, this remains computationally tractable.

B Instance Network Figures

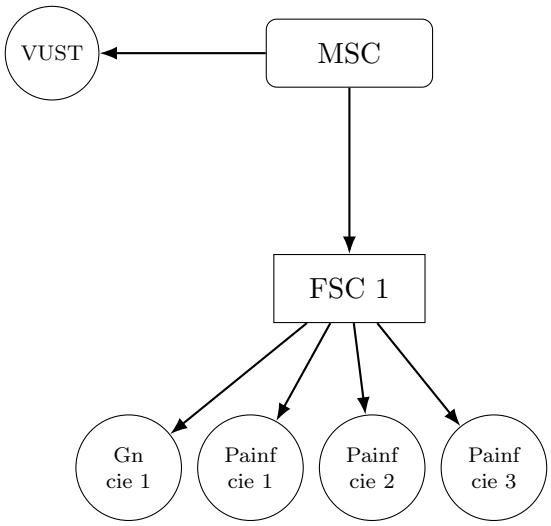

Figure 8: Logistics network for the Small instance.

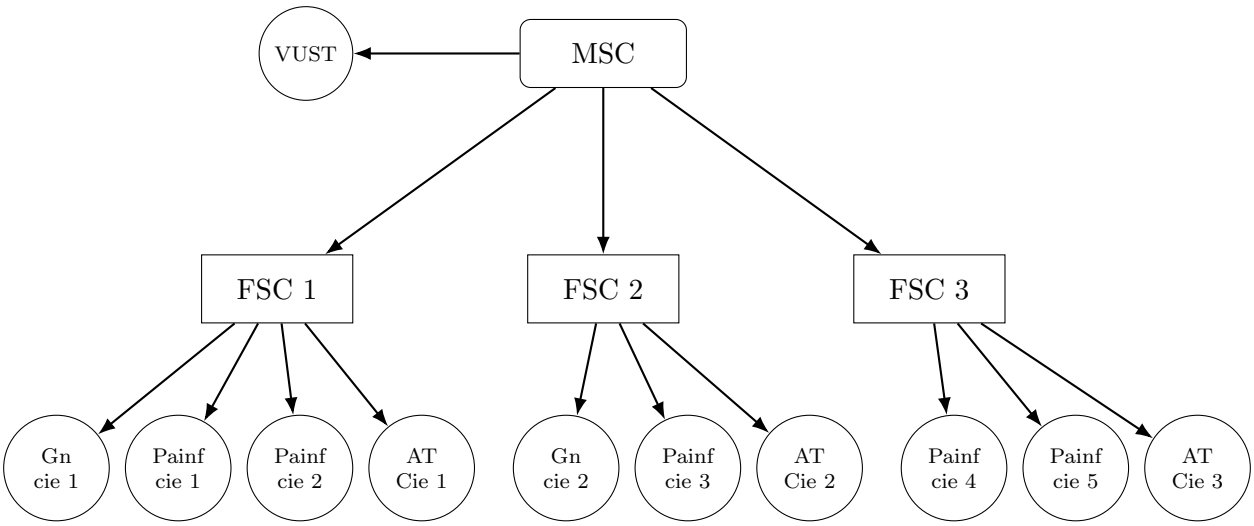

Figure 9: Logistics network for the instance Dispersed 1.

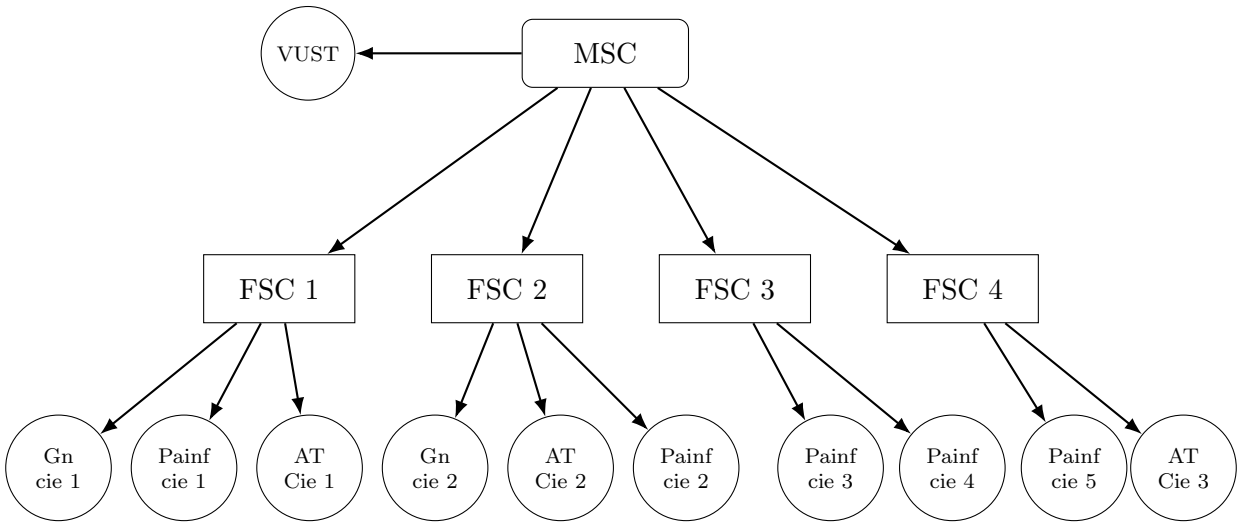

Figure 10: Logistics network the instance Dispersed 2.

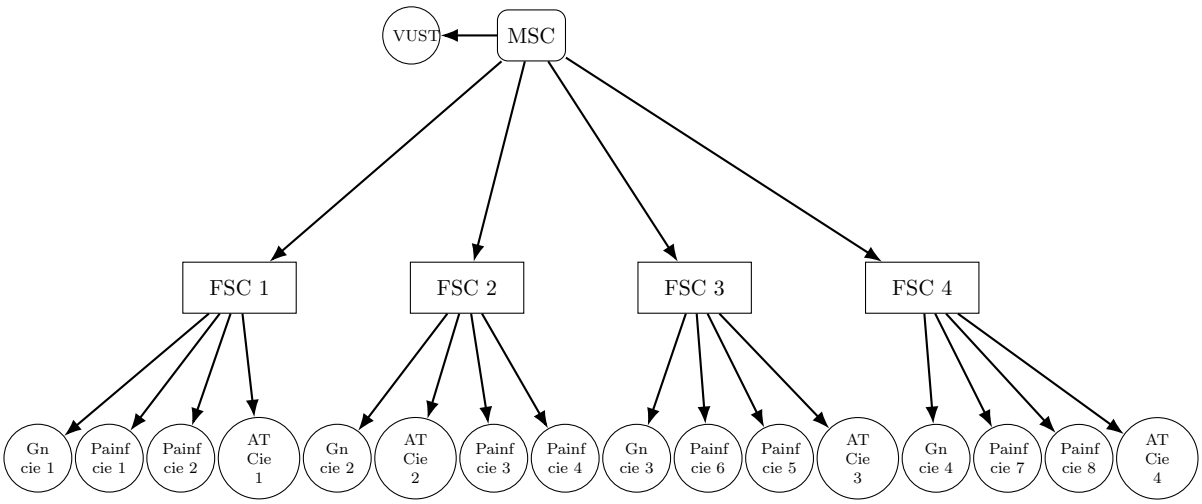

Figure 11: Logistics network for the Large instance.

C Parameter Tables

Table 11: Capacity and daily deterministic demand settings for each OU in kg.

	Food & Water		**Fuel**		**Ammo**	
OU	Capacity (kg)	Demand (kg)	Capacity (kg)	Demand (kg)	Capacity (kg)	Demand (kg)
Vust	39054	13018	128526	42842	201420	67140
Gn cie 1	18627	6209	63216	21072	131154	43718
Gn cie 2	20898	6966	73674	24558	118428	39476
Gn cie 3	20432	6500	71338	22222	119952	41000
Gn cie 4	20532	6600	72449	23333	120952	42000
Painf cie 1	24258	8086	95079	31693	174666	58222
Painf cie 2	33591	11197	78381	26127	182028	60676
Painf cie 3	28341	9447	93147	31049	172512	57504
Painf cie 4	34710	11570	100458	33486	158832	52944
Painf cie 5	31053	10351	95403	31801	167544	55848
Painf cie 6	30832	10130	93602	30000	166696	55000
Painf cie 7	30842	10140	93602	30000	166696	55000
Painf cie 8	30852	10150	93602	30000	166696	55000
Painf cie 9	30862	10160	93602	30000	166696	55000
AT cie 1	10428	3476	23763	7921	52809	17603
AT cie 2	7452	2484	23694	7898	55854	18618
AT cie 3	9099	3033	28884	9628	49017	16339
AT cie 4	9066	3000	27756	8500	49678	17000
AT cie 5	9066	3000	27756	8500	49678	17000
AT cie 6	9066	3000	27756	8500	49678	17000

Table 12: Maximum and initial storage parameters for the FSCs $f \in F$ in # CCLs.

		Initial storage											
		Gn cie				Painf cie				AT cie			
Name	Max storage	Type 1	Type 2	Type 3	Type 4	Type 1	Type 2	Type 3	Type 4	Type 1	Type 2	Type 3	Type 4
FSC 1	126	5	5	5	6	22	22	22	21	4	4	5	5
FSC 2	87	5	5	5	6	14	14	14	15	2	2	2	3
FSC 3	90	5	5	5	6	14	14	14	15	2	2	2	3
FSC 4	93	5	5	5	6	14	14	14	15	3	3	3	3

D Full Computational Results - Deterministic Setting

Table 13: Computational results for the branch-and-cut approach given deterministic demand, for each instance and T

	Small		Normal		Dispersed 1		Dispersed 2		Large	
T	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)
5	26	0,069	45	0,070	49	0,078	53	0,103	78	0,299
10	56	0,054	101	0,253	105	0,986	106	0,260	178	5,854
15	67	0,118	121	0,594	124	0,288	125	0,261	213	0,729
20	72	0,110	130	1,894	133	0,728	133	0,503	230	7,817
25	75	0,181	136	0,869	138	1,203	139	0,761	241	7,795
30	77	0,243	140	0,920	141	1,628	142	1,213	247	10,233
35	78	0,596	142	7,045	144	1,643	144	2,470	253	12,319
40	79	0,719	145	2,201	146	2,670	146	3,165	257	27,410
45	80	0,935	146	7,651	147	8,496	147	3,769	259	34,382
50	81	0,685	147	16,087	149	4,174	150	19,179	261	58,890
55	82	1,782	148	20,675	149	4,646	150	27,996	262	51,096
60	82	1,072	148	26,266	150	30,779	151	22,696	263	182,854

E Branch-and-cut with 2000-8000-0 CCL configuration up until $T = 100$

Table 14: Computational results for the branch-and-cut approach given deterministic demand, for each instance and T, and CCL configuration 2000-8000-0. (*Lower bound as time limit was reached)

	Small		Normal		Dispersed 1		Dispersed 2		Large	
T	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)
5	26	0,065	45	0,071	49	0,113	53	0,102	78	0,282
10	56	0,063	101	0,249	105	0,988	106	0,258	178	6,194
15	67	0,124	121	0,640	124	0,295	125	0,275	213	0,740
20	72	0,112	130	1,918	133	0,895	133	0,540	230	8,244
25	75	0,190	136	0,926	138	1,520	139	1,046	241	8,511
30	77	0,451	140	1,116	141	1,754	142	1,496	247	11,522
35	78	0,638	142	8,051	144	1,757	144	2,606	253	12,929
40	79	0,752	145	2,635	146	2,888	146	3,415	257	37,894
45	80	1,741	146	11,819	147	13,184	147	6,220	259	51,326
50	81	0,831	147	18,652	149	4,560	150	19,485	261	66,824
55	82	2,008	148	24,559	149	5,156	150	32,073	262	55,807
60	82	1,093	148	37,219	150	31,905	151	22,350	263	190,270
65	82	2,710	150	16,805	150	13,147	151	34,963	265	787,647
70	82	2,259	150	24,584	151	73,166	153	40,844	268	65,979
75	83	7,411	151	28,631	151	89,705	153	21,991	268	288,470
80	83	2,575	151	71,309	151	73,234	153	118,257	269	226,813
85	83	2,482	151	43,031	152	89,876	154	44,980	269	914,361
90	83	2,980	151	138,690	153	127,516	154	55,858	270	413,873
95	83	12,302	152	120,964	153	279,122	154	148,507	270	901,579
100	83	15,286	153	119,809	154	151,688	154	147,021	272	583,010

Table 15: Computational results for the multi-cut Benders decomposition given deterministic demand, for each instance and T, and CCL configuration 2000-8000-0. (*Lower bound as time limit was reached)

	Small		Normal		Dispersed 1		Dispersed 2		Large	
T	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)	Fleet size	Runtime (s)
5	26	0.17	45	0.84	50	1.51	53	1.60	78	8.88
10	56	0.86	101	3.80	105	17.48	106	11.30	178	15.48
15	67	1.25	121	7.39	124	18.04	125	19.85	213	31.40
20	72	1.47	130	16.75	133	31.35	133	28.21	230	86.84
25	75	3.69	136	33.70	138	43.38	139	241.58	241	112.17
30	77	8.62	140	47.19	142	59.87	142	52.31	247	211.03
35	78	6.90	142	224.16	144	116.04	144	129.75	253	556.46
40	79	8.49	145	89.55	146	116.11	146	122.67	257	599.28
45	80	13.00	146	123.79	147	164.14	147	319.90	259	893.46
50	81	23.38	147	139.74	149	190.02	150	375.93	261	1084.07
55	82	26.30	148	176.71	149	262.63	150	432.33	262	1746.52
60	82	20.95	148	423.76	150	319.33	151	477.39	263	2143.81

F Settings for stochastic cases.

Instance	**Service level (%)**	**Z_{min}**	**Z_{max}**	θ
Small	85	50	100	0.5
	90	50	120	0.5
	95	50	200	0.5
Normal	85	100	180	0.5
	90	100	200	1.5
	95	100	200	1.5
Large	85	250	500	2.5
	90	250	500	2.5
	95	250	500	2.5
Dispersed 1	85	100	180	1.5
	90	100	200	1.5
	95	100	200	1.5
Dispersed 2	85	100	180	1.5
	90	100	200	1.5
	95	100	200	1.5

Table 16: Table with stochastic solution methods parameters per instance and service level for the stochastic setting.